

UI-CS-AI-2026-401628201-PRJ1

پروژه شماره ۱: جستجوی ناآگاهانه و آگاهانه (۱۴۰۵/۰۲/۲۱)

[مهلت انجام: ۱۴۰۵/۰۲/۳۱]

آنچه باید پاسخ دهید:

بازی تاج و تخت: بازگشت پادشاه شب

در سرمای بی‌رحم شمال، پس از پایان بازی تاج و تخت، همه چیز تمام نشده است. دیوار فرو ریخته، ارتش مردگان شکست خورده، و به نظر می‌رسد Night King نابود شده است؛ اما در اعماق سرزمین‌های یخ‌زده‌ی آن سوی دیوار، نشانه‌هایی ظاهر می‌شود که خلاف این را ثابت می‌کند. دیده‌بان‌های شمال گزارش داده‌اند که گروهی از وایت‌ها دوباره در حال حرکت‌اند، روستاهای متروکه یکی یکی خاموش می‌شوند، و زمستانی عجیب‌تر از همیشه در راه است.

Arya Stark که سال‌ها پس از جنگ بزرگ در سفر بوده، پیامی محرمانه از Sansa Stark دریافت می‌کند: «اگر شایعات درست باشند، او بازگشته است. و این بار، فرصت اشتباه نداریم.»

طبق افسانه‌های قدیمی فرزندان جنگل، تنها راه نابودی کامل Night King استفاده از تمام قطعات باقی‌مانده‌ی Dragon Glass است که در نقاط مختلف سرزمین‌های یخی پراکنده شده‌اند. آریا باید پیش از رسیدن نایت کینگ به او، تمام این قطعات را جمع‌آوری کند.

اما مسیر ساده نیست:

- زمین‌های یخ‌زده حرکت را کند می‌کنند؛ عبور از آن‌ها انرژی بیشتری مصرف می‌کند.
- جعبه‌های متروکه و خرابه‌های باقی‌مانده از جنگ مسیر را مسدود کرده‌اند و غیرقابل عبورند.
- و بدتر از همه، Night King در همان سرزمین پرتلاطم می‌زند؛ یک برخورد کافی است تا مأموریت شکست بخورد.

مأموریت آریا روشن است:

تمام قطعات Dragon glass را جمع‌آوری کن، از مسیرهای پرهزینه‌ی یخی هوشمندانه عبور کن، از موانع دوری کن، و پیش از آنکه Night King تو را پیدا کند، شمال را یک‌بار برای همیشه نجات بده.

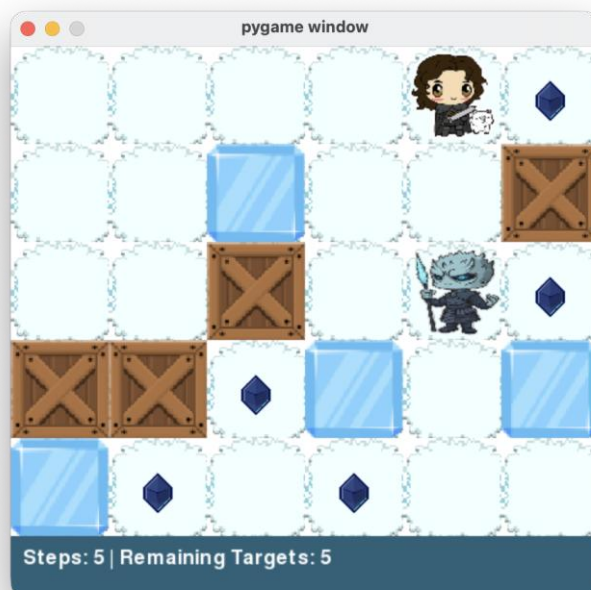
زمستان هنوز تمام نشده است.

- نکته: برای جستجوی عمق محدود می‌توانید یک حداکثر عمق معقول تخمین بزنید یا از الگوریتم *IDS* استفاده کنید.

کد اولیه پروژه را از این لینک [دانلود کنید](#).

٢. محیط

محیط بازی یک محیط شطرنجی با ابعاد $m \times n$ است که یک محیط قطعی و مشاهده‌پذیر است.



(شکل ۱: نمونه‌ای از محیط بازی که باید در آن شیشه‌های اژدها جمع‌آوری شود و از برخورد با پادشاه شب اجتناب شود)

اجزای محیط:

- آریا استارک (عامل): باید به او کمک کنید همه‌ی شیشه‌های اژدها را جمع کند و به پایگاه برگرداند.
- پادشاه شب (دشمن): بعضی مواقع پادشاه شب در زمین بازی حضور دارد. او اکثراً متحرک است و حرکت m آن از قبل قابل تعیین است.
- صخره یخی: هزینه عبور از آن ۲۰ برابر زمین عادی است.
- خرابه‌های باقی‌مانده از جنگ: عبور از آن ممکن نیست.
- شیشه‌های اژدها (اهداف): باید همه‌ی آن‌ها در محیط توسط جان اسنو جمع‌آوری شوند.

۳. مراحل پیاده‌سازی

برای تسهیل پیاده‌سازی جستجو، توابع مفیدی برای تحلیل و جستجو در محیط پیاده‌شده‌اند. این توابع بر روی اشیاء کلاس GameState عمل می‌کنند. نحوه‌ی دسترسی به آن‌ها به صورت `state.func(args)` است. فهرست توابع تعامل با محیط که در اختیار شما قرار می‌گیرند:

۱. `get_grid_size()->tuple`: ابعاد نقشه را برمی‌گرداند.
۲. `get_enemy_position()->tuple`: موقعیت فعلی پادشاه شب (دشمن) را برمی‌گرداند.
۳. `get_enemy_next_position()->tuple`: موقعیت بعدی دشمن در مسیر حرکت را به صورت یک tuple برمی‌گرداند.
۴. `get_enemy_path()->list`: مسیر کامل حرکت دشمن را برمی‌گرداند.
۵. `get_agent_position()->tuple`: موقعیت فعلی جان اسنو (عامل) را برمی‌گرداند.
۶. `get_terrain_cost(pos)->float`: هزینه حرکت بر روی یک سلول خاص را با در نظر گرفتن نوع زمین (برف عمیق و...) و برخورد با دشمن محاسبه می‌کند.
۷. `get_targets_positions()->list`: موقعیت‌های تمام شیشه‌های اژدها (اهداف) موجود در بازی را برمی‌گرداند.
۸. `get_icees_positions->list`: موقعیت‌های تمام توده‌های برف عمیق (بوته‌ها) را برمی‌گرداند.
۹. `is_crates_position(position)->bool`: مشخص می‌کند آیا در این موقعیت صخره یخی (مانع) است.
۱۰. `get_crates_positions()->list`: موقعیت‌های تمام صخره‌های یخی را برمی‌گرداند.
۱۱. `is_icees_position(position)->bool`: مشخص می‌کند آیا در این موقعیت توده برف عمیق است.
۱۲. `is_goal_state()->bool`: بررسی می‌کند آیا تمام شیشه‌های اژدها جمع‌آوری شده‌اند یا خیر.
۱۳. `is_collision_state()->bool`: بررسی می‌کند آیا جان اسنو با پادشاه شب برخورد کرده است یا خیر.
۱۴. `get_successors()->list`: حالت‌های ممکن بعدی را با در نظر گرفتن حرکات مجاز تولید می‌کند. خروجی شامل یک لیست از ترکیب کنش، هزینه، و موقعیتی که به آن منجر می‌شود خواهد بود.

نکته: در صورتی که در آرگومان تابع عبور از دیوار $True$ شود، $get_successors(toward_walls=True)$ حرکت به سمت صخره‌ها به عنوان حرکت مجاز در نظر گرفته می‌شود و به عنوان کنش‌های مجاز برمی‌گردد. در این محیط اگر عامل به سمت مانع یا خارج از محیط کنش انجام دهد، در وضعیت حاصل از آن کنش موقعیت عامل تغییر نخواهد کرد و فقط $step$ تغییر می‌کند.

تابع اکتشافی مورد استفاده در الگوریتم A^* در ابتدای تعریف می‌شود که باید آن را پیاده‌سازی کنید. در پیاده‌سازی این تابع می‌توانید از توابع تعامل با محیط که در اختیار شما قرار گرفته است استفاده کنید.

```

1 def astar(initial_state):
2     def heuristic(*args):
3         # Implement heuristic here
4         return h
5
6     # Implement A* here
7     return actions
    
```

مثالی از ساختار تابع A^*

توجه کنید که توابع پیاده‌سازی شده باید بتوانند در نقشه‌های مختلف به کار گرفته شوند. نمونه‌هایی که می‌توانند از نظر ابعاد محیط، تعداد اهداف و موقعیت اولیه آریا استارک و حرکت پادشاه شب متفاوت باشند. به همین دلیل سه محیط اولیه در اختیار شما قرار داده شده است.

۴. سیستم امتیازدهی محیط

در پایان اجرای هر الگوریتم، گزارشی در ترمینال نمایش داده می‌شود که شامل نام نقشه، زمان اجرای جستجو، امتیاز نهایی، هزینه، تعداد گره‌های بسط داده‌شده، و قدم‌های برداشته‌شده توسط عامل است. امتیاز نهایی عامل براساس هزینه حرکت‌ها، جمع‌آوری آیتم‌ها و تعامل با دشمن محاسبه می‌شود.

هزینه‌های حرکت:

عامل هنگام حرکت در محیط، بسته به نوع خانه‌ای که وارد آن می‌شود هزینه متفاوتی پرداخت می‌کند:

- حرکت روی خانه‌های عادی دارای هزینه پایه است.
- عبور از خانه‌های برفی هزینه بیشتری دارد.
- عبور از خانه‌های یخی بسیار پرهزینه‌تر است.

- برخورد با Night King جریمه سنگینی دارد.
 - در صورتی که تعداد حرکات از حد مشخصی بیشتر شود، اجرای بازی متوقف خواهد شد.
- منطق مربوط به این هزینه‌ها در فایل‌های مربوط به محیط پیاده‌سازی شده است و می‌توانید برای مشاهده جزئیات آن به بخش‌های زیر مراجعه کنید:

- `env\constants.py`
- `env\score.py`

پاداش‌ها:

عامل در شرایط زیر پاداش دریافت می‌کند:

- جمع‌آوری هر قطعه Dragon Glass
 - جمع‌آوری تمام اهداف موجود در نقشه
 - جمع‌آوری سلاح ویژه (در صورت وجود)
 - از بین بردن Night King (در بخش اختیاری)
- مقادیر دقیق هر پاداش نیز در فایل `env/constants.py` قابل مشاهده است.

نحوه محاسبه Score:

پس از پایان اجرای هر الگوریتم، محیط به صورت خودکار براساس موارد زیر امتیاز نهایی را محاسبه می‌کند:

- تعداد اهداف جمع‌آوری شده
- تعداد حرکت‌های انجام شده
- هزینه کلی مسیر
- برخورد با دشمن
- دریافت سلاح
- حذف دشمن (در صورت پیاده‌سازی قابلیت اضافه)

دانشجویان نیازی به پیاده‌سازی سیستم امتیازدهی ندارند، اما بهتر است برای طراحی heuristic بهتر و تحلیل عملکرد الگوریتم‌ها، منطق امتیازدهی محیط را بررسی کنند.

۵. نحوه اجرا

فایل `main.py` را بررسی کنید. طبق این فایل با استفاده از تابع `play` که در `env\__init__.py` پیاده‌سازی شده است، محیط و الگوریتم شما اجرا می‌شود.

الگوریتم‌های شما در فایل‌های `bfs`، `dls`، `ucs`، `a_star` باید پیاده‌سازی شوند. تمامی این فایل‌ها در پوشه‌ی `search` هستند. هیچ‌گونه محدودیتی بجز عدم امکان تغییر در منطق و امتیازهای محیط وجود ندارد. شما می‌توانید خلاق باشید. نقشه‌های محیط در `env\maps` هستند. نام آن‌ها را بررسی کنید و در تابع `play` برای بررسی صحت الگوریتم خود استفاده کنید.

۶. نحوه ارزیابی

هر بار پس از اجرای محیط، یک گزارش در ترمینال چاپ می‌شود که نشان‌دهنده امتیاز کسب شده، زمان اجرا و تعداد گره‌های بسط داده شده است. ارزیابی کیفیت تابع اکتشافی پیاده‌سازی شده براساس تعداد گره‌های بسط داده شده توسط الگوریتم جستجوی A^* خواهد بود، که در جدول زیر امتیاز مرتبط برای محیط‌های *easy*، *medium* و *hard* نشان داده شده است:

امتیاز	گره‌های بسط داده شده در نقشه سخت	گره‌های بسط داده شده در نقشه متوسط	گره‌های بسط داده شده در نقشه ساده
۱۰۰٪	کمتر از ۲۵۰۰	کمتر از ۵۰۰	کمتر از ۱۰۰
۸۰٪	کمتر از ۲۰۰۰۰	کمتر از ۱۰۰۰۰	کمتر از ۱۵۰
۵۰٪	کمتر از ۱۵۰۰۰۰	کمتر از ۵۰۰۰۰	کمتر از ۲۰۰

نکته: اجرای الگوریتم‌های *BFS*، *DLS*، *UCS* روی محیط *hard* اجباری نیست.

همانطور که قبلاً گفته شد در ارزیابی پیاده‌سازی انجام شده از نمونه محیط‌های دیگری نیز استفاده می‌شود که انتظار می‌رود تعداد گره‌های بسط داده شده در حد قابل قبول باشد. بنابراین علاوه بر صحت پیاده‌سازی الگوریتم‌های قبلی در فایل‌های به نام خودشان، تابع اکتشافی پیاده‌سازی شده نیز از نظر تعداد گره‌های بسط داده شده مورد بررسی قرار خواهد گرفت.

۷. قابلیت اضافه (امتیازی): قتل پادشاه شب

در برخی از نقشه‌های این پروژه نیز سلاح ویژه‌ای در محیط قرار داده شده است. این سلاح نمایانگر *Valyrian Dagger* است که آریا می‌تواند آن را از محیط جمع‌آوری کند.

قوانین این بخش

- برخی نقشه‌ها شامل یک *weapon* هستند.
- آریا می‌تواند مشابه سایر آیتم‌ها به موقعیت سلاح حرکت کرده و آن را بردارد.
- پس از برداشتن سلاح:
 - برخورد با *Night King* دیگر به معنای شکست نیست.
 - در صورت رسیدن آریا به خانه‌ی نایت کینگ، دشمن از بین می‌رود.
 - پس از کشته شدن نایت کینگ:
 - دشمن دیگر در محیط حرکت نخواهد کرد.
 - جریمه برخورد با دشمن حذف می‌شود.
- دانشجویانی که بتوانند مکانیزمی طراحی کنند که عامل به صورت هوشمند تصمیم بگیرد:
 - چه زمانی رفتن به سمت سلاح به صرفه است

- آیا کشتن نایت کینگ باعث کاهش هزینه کلی مسیر می شود
 - و سپس در صورت مناسب بودن، سلاح را جمع آوری کرده و دشمن را حذف کند
 - امتیاز اضافه دریافت خواهند کرد.
- نکته: اگر رفتن به سمت سلاح باعث افزایش غیرمنطقی هزینه مسیر شود، عامل نباید صرفاً همیشه سلاح را انتخاب کند. بخشی از ارزیابی این قسمت مربوط به تصمیم گیری صحیح عامل است.

آنچه باید تحویل دهید:

- کد پروژه
 - گزارش از پروژه به صورت pdf (دقت فرمایید که حتما لینک پابلیک گیتهاب پروژه در این گزارش ذکر شده باشد)
- را در سامانه کوئرا بارگزاری کنید. (یک نفر از هر گروه کافی است).

هشدار: لطفاً انجام پروژه را به روز پایان ددلاین موکول نکنید!

تندرست و پیروز باشید

فریاد نصیری منجم